

ROOTOPATHY

CareOS

Complete Build Specification
Java Spring Boot + React/Node Edition

Start-from-scratch product, architecture, data, security, implementation
and Codex execution baseline

Version 1.0 - 13 September 2026

One Patient. One Complete Story. One Coordinated Plan.

No patient data, passwords, API keys or production credentials are included.

Document status and authority

This document defines a new Java Spring Boot + React/Node implementation of CareOS from first commit through production readiness. It is a build specification, not production code and not a claim that the existing Laravel application has been converted.

Item	Decision
Target architecture	Java backend with Spring Boot; React + TypeScript web UI built with Node tooling
Delivery model	Modular monolith first, with explicit extraction boundaries for AI, FHIR and proven high-load domains
Database	PostgreSQL with forced row-level security and migration-controlled invariants
Protected visual baseline	Existing CareOS Design System and COS-01 through COS-27 must be supplied and preserved
Patient safety	Clinician-in-the-loop; AI never produces an unreviewed final clinical decision
Use	Authoritative architecture baseline; each module still requires approval of detailed mockups and policy decisions before implementation

Important: A complete code ZIP cannot be produced safely from this PDF alone. Codex must also receive the original COS assets, Module 1 and Module 2 approved mockups/prototypes, exact design tokens, policy decisions and any reusable data contracts. This PDF tells Codex how to use and protect them.

Authority order

1. Explicit product-owner decisions and approved clinical-safety policies.
2. This Java edition for architecture and start-from-scratch implementation rules.
3. Approved per-module high-fidelity mockups and screen registers.
4. COS-01 through COS-27 for clinical visual/workflow truth.
5. Checked-in OpenAPI, database migrations, permission registry and audit-event registry.
6. Tests and freshly verified repository behavior.

Required input package before coding

Required item	Why
This PDF	Architecture and execution baseline
COS-01 through COS-27 source assets	Clinical workflow and UI preservation
M1-01 through M1-23 approved mockups	Administration workspace fidelity
M2-01 through M2-29 final mockups/prototype	Workforce workflow fidelity
CareOS Design System 1.0 tokens/assets	No arbitrary visual choices
Approved policy decision register	Retention, consent, working age, break-glass, notifications and AI

1. Executive product definition

1.1 Vision

CareOS is the clinical and operational system of record for ROOTOPATHY: a multi-tenant platform that connects one patient story, a governed workforce, coordinated care delivery, measurable outcomes and accountable AI assistance.

1.2 Core promise

One Patient. One Complete Story. One Coordinated Plan.

1.3 Non-negotiable principles

- Patient before modality; safety before novelty.
- Assessment before intervention; prioritize before integrate.
- Evidence honesty, explicit uncertainty and no unsupported root-cause claims.
- Red-flag escalation, respect for professional scope and accountable clinical ownership.
- Do not accumulate therapies without purpose, owner, monitoring and stop criteria.
- AI assists; an authorized RCP/doctor reviews and approves clinically relevant outputs.
- Every important state change is authorized, transactional, versioned and auditable.
- Tenant privacy is enforced in application logic and the database.

1.4 In-scope capabilities

Domain	Capability
Administration	Organization, facilities, departments, locations, hours, services, identifiers and governance
Workforce	Employees, practitioners, credentials, scope, assignments, access, availability and lifecycle
Patients	Identity, consent, caregiver/proxy, duplicate resolution and longitudinal record
Care delivery	Appointments, assignments, encounters, assessments, documents, plans and follow-up
AI	Purpose-limited extraction, summarization and suggestions with clinician review
Operations	Billing, payments, reporting, exports, notifications, audit and configuration history
Interoperability	FHIR, labs, imaging, WhatsApp, payments, calendar and mobile APIs

1.5 Explicit non-goals for the initial release

- No autonomous diagnosis, prescribing or emergency disposition.
- No big-bang microservice architecture.
- No duplicate authentication tables for doctors, employees or patients.
- No public clinical or credential document URLs.
- No cross-organization identity disclosure without approved lawful basis.
- No production use of synthetic scanner, self-approval or test data.

2. Complete module map and dependency order

Module	Workspace	Baseline screens	Depends on
M1	Administration	M1-01 to M1-23	Identity/tenant foundation
M2	Workforce	M2-01 to M2-29	M1
M3	Patient Registry	P3-01 to P3-16	M1 + identity
M4	Appointments	P4-01 to P4-15	M1 + M2 + M3
M5	Encounters	P5-01 to P5-12	M2 + M3 + M4
M6	Clinical Assessment	COS-01 to COS-27	M5
M7	Documents and Results	P7-01 to P7-11	M3 + M5
M8	AI Assistance	P8-01 to P8-10	M5 + M6 + M7
M9	Care Planning	P9-01 to P9-12	M6 + M8
M10	Follow-up and Outcomes	P10-01 to P10-09	M9
M11	Billing	P11-01 to P11-11	M3 + M4 + M5
M12	Reporting and Governance	P12-01 to P12-10	All implemented modules
M13	Integrations	P13-01 to P13-10	Stable internal APIs

2.1 Build order

1. Foundation: repository, Design System, identity, tenancy, security, PostgreSQL RLS, audit/outbox and test isolation.
2. Module 1 administration and organization activation.
3. Module 2 workforce and practitioner readiness.
4. Module 3 patient identity and consent.
5. Module 4 appointments and eligible clinician assignment.
6. Module 5 encounter/episode record.
7. Module 6 implementation of protected COS-01 through COS-27.
8. Module 7 documents/results, then Module 8 AI assistance.
9. Modules 9-10 care plan, follow-up and outcomes.
10. Module 11 billing, Module 12 reporting/governance, Module 13 integrations.

2.2 Release rule

Each module moves through: architecture -> policy decisions -> high-fidelity clickable prototype -> implementation specification -> production implementation -> full verification -> owner approval -> freeze tag. A later module must not silently change a frozen module.

3. Final Java + React/Node technology baseline

Layer	Baseline	Decision
JDK	Java 25 LTS	Use a production OpenJDK distribution; pin image digest
Backend	Spring Boot 4.1.x	Pin current compatible patch through Maven wrapper/lock controls
Spring	Spring Framework 7.x, Spring Security, Data JPA/JDBC, Validation, Actuator	Use only required starters
Build	Maven Wrapper	Reproducible builds; parent BOM controls versions
Frontend	React 19.3 + strict TypeScript	Authenticated SPA; no SEO dependency
Node	Node 24 LTS	Frontend build/test toolchain, not the clinical backend
Bundler	Vite compatible stable	Pin exact version in package lock
Styling	Tailwind compatible stable + CSS variables	CareOS Design System 1.0 tokens are authoritative
Database	PostgreSQL 18.x	Managed production service; forced RLS
Cache/jobs	Redis/Valkey 8.x compatible	Cache, rate limits, sessions, short jobs and coordination
Storage	S3-compatible private object storage	Quarantine and protected buckets
API	REST/JSON + OpenAPI 3.1 + RFC 9457	Stable web/mobile/integration contracts
Containers	Docker Compose local; OCI production images	Separate API, worker and scheduler processes

Verified on 13 September 2026: Spring Boot documentation lists 4.1.1 as stable and Java compatibility through 26; Java 25 is an LTS release; Node 24 is LTS; React documentation identifies 19.3; PostgreSQL 18 is current. Patch versions must be updated through tested dependency-renovation pull requests.

3.1 Browser authentication architecture

- Same-origin secure HttpOnly session cookie for the web workspace.
- CSRF protection on every state-changing browser request.
- Never store access tokens in localStorage.
- Future mobile/integration clients use OAuth 2.1/OIDC Authorization Code with PKCE or client credentials as appropriate.

3.2 Why modular monolith first

- One transactional boundary for clinical, authorization, audit and outbox invariants.
- Lower deployment and operational cost while modules are evolving.
- Explicit module APIs and architecture tests preserve future extraction options.

4. Repository and module structure

```
careos-java/  
backend/  
pom.xml  
src/main/java/com/rootopathy/careos/  
shared/ identity/ tenancy/ administration/ workforce/ patient/  
scheduling/ encounter/ assessment/ documents/ ai/ careplan/  
outcomes/ billing/ reporting/ integration/  
src/main/resources/  
application.yml  
db/migration/  
src/test/  
frontend/  
package.json package-lock.json vite.config.ts tsconfig.json  
src/  
app/ routes/ layouts/ pages/ features/ entities/ shared/  
design-system/ api/ auth/ test/  
contracts/  
openapi/ events/ permissions/ audit-events/ schemas/  
docs/  
adr/ architecture/ modules/ security/ operations/ traceability/  
deploy/  
docker/ compose/ kubernetes-or-platform/ observability/  
scripts/  
compose.yaml  
.env.example  
README.md  
AGENTS.md
```

4.1 Backend package rule

Every business module contains domain, application, infrastructure and api packages. Domain code cannot import infrastructure or web classes. Cross-module reads use published application interfaces or read models. Cross-module mutations use commands/events, not direct repository writes.

Package	Responsibility
domain	Aggregates, value objects, invariants, state transitions and domain events
application	Use cases, command/query handlers, transactions, authorization orchestration and DTOs
infrastructure	PostgreSQL repositories, object storage, Redis, messaging and external adapters
api	REST controllers, request validation, problem details and mapping

4.2 Frontend architecture rule

Use feature-oriented React modules with generated API types. Pages compose domain features; shared components contain no CareOS business rules. Server data uses a query library with tenant-safe cache keys. Forms use schema validation for usability but server validation remains authoritative.

4.3 Architectural enforcement

- ArchUnit verifies allowed backend dependencies.
- ESLint boundaries verify frontend feature layering.
- OpenAPI generation/validation prevents undocumented routes.
- Permission and audit-event registries fail builds on duplicates or undefined references.

5. Runtime topology and scaling

Component	Initial production	Scale path
Edge	TLS reverse proxy/WAF/load balancer	CDN for public assets; rate and bot controls
Web	React static bundle served behind same origin	CDN versioned assets
API	At least two stateless Spring Boot instances	Horizontal autoscaling
Workers	Dedicated Spring worker processes	Scale by queue depth and latency
Database	Managed PostgreSQL primary	Pooling, tuning, replicas, partitioning when measured
Redis	Managed Redis/Valkey	Separate cache/session/job prefixes or clusters
Storage	Private object storage	Lifecycle tiers and regional replication
AI	Separate Python service	Independent model/provider/GPU scaling
FHIR	Integration boundary	Optional Java HAPI FHIR service

5.1 Statelessness

- No local-disk dependency for user documents or durable job state.
- Sessions stored in Redis or an approved identity provider; encryption keys externalized.
- Every retryable request uses an idempotency key and stable result record.

5.2 Proposed service objectives

Metric	Initial target	Notes
Availability	99.9% monthly	Excludes approved maintenance; owner approval required
API read latency	p95 under 400 ms	Excludes reports, file transfer and AI
API write latency	p95 under 800 ms	Long work is queued
Interactive search	p95 under 1 second	Indexed server search
RPO	15 minutes	Confirm with chosen database platform
RTO	4 hours initially	Exercise through restore drills

5.3 When to extract a service

Extract only after a measured scaling, security, deployment cadence or specialist-technology need. AI inference and FHIR are pre-approved boundaries; all other extraction requires an ADR, data ownership, event contract, failure model and operational budget.

6. Tenant, identity and access model

6.1 Identity entities

Entity	Meaning	Rule
users	Authenticated account	Only authentication table
person_profiles	Human identity/contact profile	Can exist without user
organization_person_links	Tenant-visible person association	Prevents global browsing
workforce_members	Organization-specific staff identity	Can activate without user account
patient_profiles	Patient identity	Separate from workforce and user
caregiver_relationships	Authorized proxy relationship	Effective dated and revocable
organization_memberships	User membership in organization	Independent of employment
access_assignments	Role plus resource scope	Independent of clinical scope

6.2 Tenant context

- Organization is selected after authentication and verified against active membership.
- Every request establishes actor, organization, purpose and correlation ID.
- Tenant-scoped SQL runs only inside a transaction that sets PostgreSQL context with SET LOCAL.
- Connection-pool checkout must not retain a previous tenant context.
- Routes bind organization and resource together; guessed cross-tenant UUIDs return hidden/not-found or forbidden according to policy.

6.3 Baseline workforce roles

#	Role
1	Platform Administrator
2	Organization Owner
3	Organization Administrator
4	Workforce Administrator
5	HR Administrator
6	Facility Administrator
7	Credentialing Officer
8	Clinical Governance Approver
9	Practitioner
10	Clinical Support Staff
11	Reception/Front Desk
12	Billing/Accountant
13	Organization IT Support
14	Configuration Auditor
15	System Service Account

6.4 Access rules

- Least privilege and scope at organization, facility, department or location.
- No actor can assign a role or scope higher than their own delegable authority.
- Final active Organization Owner and final privileged workforce administrator safeguards exist in policy, transaction and database layers.

- IT support has no automatic clinical access; production elevation is time-limited, reasoned, MFA-protected and audited.

7. Authentication, invitations and MFA

Flow	Requirements
Login	Email/identifier plus password, rate limiting, session rotation, risk logging
MFA	TOTP initially, recovery codes, enrollment evidence, step-up challenge
Invitation	One-use random token, hash stored, expiry, resend rotates, revoke invalidates
Password reset	Short-lived one-use token, session revocation option, audit
Organization selection	Only active memberships; active tenant stored server-side
Recent authentication	Required for high-risk lifecycle, access, export and approval actions
Service accounts	No interactive login; purpose-specific credentials and rotation

7.1 Account-linking rule

Creating a patient or workforce member never automatically creates a user. Account linkage occurs through an authorized existing-user match or an invitation. One user may have organization memberships and may be linked to a workforce or patient profile only through explicit governed relationships.

7.2 Session security

- Secure, HttpOnly, SameSite cookies; regenerate on authentication and privilege change.
- Idle and absolute session expiry; shorter privileged session policy.
- CSRF tokens and Origin/Referer validation for browser mutations.
- Revoke sessions after password reset, account suspension or offboarding as policy requires.
- Never log passwords, MFA seeds, recovery codes or invitation tokens.

8. PostgreSQL data rules and RLS

8.1 Standard columns

Column	Rule
id	UUIDv7 domain identifier
organization_id	Non-null on tenant-owned rows; composite FK component
created_at/created_by	UTC instant and actor/service identity
updated_at/updated_by	UTC instant and actor for mutable drafts/current records
lock_version	Integer optimistic-lock revision
effective_from/effective_to	Half-open effective interval; UTC timestamps or dates by domain
status	Constrained state-machine value
supersedes_id	Historical version lineage where applicable
reason	Mandatory for governed changes

8.2 RLS implementation

```
BEGIN;  
SET LOCAL app.current_organization_id = :organizationId;  
SET LOCAL app.current_actor_id = :actorId;  
SET LOCAL app.current_purpose = :purpose;  
-- all tenant queries and mutations occur here  
COMMIT;
```

- ENABLE ROW LEVEL SECURITY and FORCE ROW LEVEL SECURITY on every tenant table.
- Application database role is not the table owner and cannot bypass RLS.
- Policies cover SELECT, INSERT, UPDATE and DELETE separately.
- Background jobs establish tenant and actor/service context before querying.
- Exports and signed URLs reapply the same policy and purpose checks.
- Automated tests attack guessed UUIDs and cross-tenant relationships at SQL and HTTP levels.

8.3 Database invariants

- Composite foreign keys maintain organization consistency.
- GiST/exclusion constraints prevent prohibited effective-date overlaps.
- Triggers protect append-only audit/evidence and maker-checker/final-owner invariants.
- No cascade delete from organization, workforce, patient or encounter into historical evidence.

9. Audit, outbox and configuration versioning

9.1 Audit event

Field	Requirement
event_id	Globally unique UUIDv7
event_name	Canonical registry key; no arbitrary strings
organization_id	Tenant where applicable
actor	User/service and effective role/scope
subject/resource	Typed identifier and safe display reference
occurred_at	UTC authoritative timestamp
reason/purpose	Required for governed/sensitive actions
correlation_id	Trace request, transaction and child changes
change_summary	Minimum necessary before/after metadata; no secrets
policy evidence	MFA, recent-auth and maker-checker evidence when required

9.2 Transactional outbox

The domain mutation, audit event and outbox message commit in the same PostgreSQL transaction. A publisher claims rows safely, publishes versioned events, records attempts and retries idempotently. Consumers deduplicate by event ID.

9.3 Configuration batching

One logical operation creates exactly one parent configuration version/change request. Individual resources are child items. A weekly availability or operating-hours edit must never create one parent version per weekday.

9.4 Event governance

- Registry test fails on duplicate, undefined or unused required events.
- Event names describe completed outcomes, not contradictory command/result pairs.
- Payload schema versions are backward compatible or explicitly migrated.

10. Document and credential security pipeline

Stage	Behavior
Upload intent	Authorize tenant, purpose, type and size; issue opaque upload reference
Private staging	Receive object in non-public quarantine storage
Validation	MIME, magic bytes, size, extension, SHA-256 and integrity checks
Scanning	Asynchronous malware adapter; bounded retry; production fails closed
Promotion	Only clean evidence moves to protected private storage
Rejection	Infected/invalid evidence isolated; user-safe error; incident evidence
Preview/download	Short-lived signed intent after tenant, purpose and sensitivity authorization
Retention	Jurisdiction/policy schedule, legal hold and auditable disposal

10.1 Prohibited behavior

- No public bucket or permanent document URL.
- No preview while quarantined, queued, scanning, infected or failed.
- No object keys, signed URLs or tokens in logs, audit or outbox payloads.
- No synthetic malware scanner outside local/test profiles.
- No file content sent to AI without separate purpose and authorization.

10.2 Storage abstraction

Define DocumentStorage, MalwareScanner, SignedAccessService and RetentionPolicy ports. Provide local/test adapters and production S3/scanner adapters. Unconfigured production adapters fail startup or fail the operation closed.

11. Module 1 detailed screen specification

ID	Screen	Required production behavior
M1-01	Login	Authenticate, throttling, password recovery, safe redirect
M1-02	Invitation	Validate one-use token, confirm identity, set password
M1-03	MFA	Enroll/challenge TOTP, recovery codes, verified state
M1-04	Organization selector	Choose only authorized active organization
M1-05	Admin dashboard	Readiness, counts, exceptions, lifecycle status
M1-06	Setup checklist	Server-calculated gates and deep links
M1-07	Organization profile	Legal/display identity and governance status
M1-08	Registration/identifiers	Registration numbers and schemes
M1-09	Addresses/contacts	Effective contact and address records
M1-10	International settings	Country, timezone, locale, date/number rules
M1-11	Governance contacts	Clinical, privacy, security and billing contacts
M1-12	Facilities	Server list/filter; one Add facility action
M1-13	Facility wizard	Draft facility, hierarchy, address and activation
M1-14	Departments/units	Hierarchy, parent department and effective lifecycle
M1-15	Locations	Facility-bound physical/virtual locations
M1-16	Operating hours	Weekly intervals, holidays, overnight periods, one batch
M1-17	Service catalogue	Governed service definitions
M1-18	Facility services	Effective service availability by facility/location
M1-19	Identifier schemes	Versioned prefix/pattern/sequence rules
M1-20	Administrator access	Invite, scope and lifecycle; final owner guard
M1-21	Review/activate	Inline run ID/time/gates; submit and independent approval
M1-22	Configuration history	Filter, compare, cursor page and purpose export
M1-23	Audit log	Server filters, event detail and purpose-bound export

11.1 Organization hierarchy

One ROOTOPATHY organization can operate many centers. Each center is a facility; facilities contain departments and locations. Services and operating hours are assigned at appropriate levels. The organization selector supports future multi-organization or platform users without forcing duplicate organization profiles.

12. Module 1 entities and route families

Domain	Core tables/entities
Identity	users, mfa_methods, recovery_codes, invitations, sessions
Tenancy	organizations, organization_memberships, roles, permissions, role_permissions, access_assignments
Organization	organization_profiles, identifiers, addresses, contacts, international_settings, governance_contacts
Network	facilities, departments, locations, operating_hours, holiday_exceptions
Services	services, facility_services, location_services
Identifiers	identifier_schemes, scheme_versions, sequences, issued_identifiers
Governance	configuration_versions, change_requests, change_items, activation_runs, activation_results
Evidence	audit_events, outbox_events, idempotency_records, export_jobs

12.1 API families

```
/api/v1/auth/*  
/api/v1/organizations  
/api/v1/organizations/{orgId}/profile|contacts|settings|governance-contacts  
/api/v1/organizations/{orgId}/facilities|departments|locations|operating-hours  
/api/v1/organizations/{orgId}/services|facility-services  
/api/v1/organizations/{orgId}/identifier-schemes  
/api/v1/organizations/{orgId}/administrators|invitations  
/api/v1/organizations/{orgId}/activation-runs|activation-requests  
/api/v1/organizations/{orgId}/configuration-history|audit-events|exports
```

12.2 Key validation

- New facility initial status is server-forced to draft.
- Closure reason, closed-from and expected reopening appear and validate only for closure/suspension.
- Department parent must be a department in the same organization/facility hierarchy.
- Operating intervals reject overlap and express overnight explicitly.
- Activation requires a fresh successful validation run and independent approver in production.

13. Module 2 detailed screen specification

ID	Screen	Required production purpose
M2-01	Workforce dashboard	Metrics, exceptions and work queues
M2-02	Workforce directory	Server search/filter/sort/cursor page
M2-03	Add workforce member	Persist pathway and access intent
M2-04	Duplicate/person search	Organization-scoped match and decision
M2-05	Personal and contact information	Identity/contact proposal
M2-06	Engagement/employment details	Effective engagement and manager
M2-07	Practitioner profile	Profession and practitioner lifecycle
M2-08	Qualifications	Create/correct/supersede evidence
M2-09	Professional registrations and licences	Create/renew/suspend/revoke history
M2-10	Credential document upload	Private quarantine upload and scan
M2-11	Credential verification queue	Filtered verification queue and SLA
M2-12	Credential review detail	Clean evidence preview and independent decision
M2-13	Specialties	Primary/secondary effective specialties
M2-14	Scope of practice	Versioned scope and independent approval
M2-15	Organization/facility/department/location assignments	Validated hierarchy and effective assignments
M2-16	Practitioner service assignments	Service eligibility and lifecycle
M2-17	Application roles and permissions	Existing governed RBAC assignment only
M2-18	Availability/working pattern	Atomic weekly intervals and exceptions
M2-19	Invitation and account access	Link existing user or invite
M2-20	Review and activate	Readiness, submit and independent activation
M2-21	Workforce member profile	Canonical selected-member profile
M2-22	Edit/transfer assignment	Transactional transfer/end with impact
M2-23	Suspend/reactivate	Governed suspension/reactivation
M2-24	Offboarding	Coordinated historical offboarding
M2-25	Expiring credentials dashboard	Expiry buckets, reminders and escalation
M2-26	Workforce configuration history	Version list, compare and detail
M2-27	Workforce audit log	Filter, detail and purpose export
M2-28	Controlled registries	Versioned governed catalogues; reuse RBAC
M2-29	Lifecycle and evidence timeline	Correlated member evidence timeline

14. Module 2 workflows and exact schema baseline

14.1 Practitioner onboarding

M2-01 -> 02 -> 03 -> 04 -> 05 -> 06 -> 07 -> 08 -> 09 -> 10 -> 11/12 independent verification -> 13 -> 14 independent scope approval -> 15 -> 16 -> 17 -> 18 -> optional 19 -> 20 independent activation -> 21.

14.2 Non-clinical onboarding

M2-03 -> 04 -> 05 -> 06 -> 15 -> 17 -> 18 -> optional 19 -> 20 -> 21. Practitioner-only requirements are excluded by the server-derived workflow plan.

14.3 Forty-four Module 2 tables

#	Table	#	Table
1	person_profiles	2	person_profile_aliases
3	person_contacts	4	person_addresses
5	organization_person_links	6	person_match_keys
7	person_merge_requests	8	workforce_members
9	workforce_identifiers	10	employment_engagements
11	practitioner_profiles	12	professional_registrations
13	qualifications	14	practitioner_specialties
15	practitioner_credentials	16	credential_documents
17	credential_scan_attempts	18	credential_verifications
19	scope_definitions	20	scope_requirements
21	scopes_of_practice	22	scope_activities
23	scope_restrictions	24	workforce_assignments
25	practitioner_service_assignments	26	availability_profiles
27	availability_periods	28	availability_exceptions
29	access_assignment_scopes	30	workforce_readiness_runs
31	workforce_readiness_results	32	workforce_activation_requests
33	workforce_offboarding_requests	34	workforce_lifecycle_transitions
35	workforce_configuration_snapshots	36	workforce_configuration_change_requests
37	workforce_configuration_change_items	38	workforce_export_jobs
39	credential_legal_holds	40	practitioner_eligibility_evidence
41	workforce_notification_deliveries	42	workforce_registry_definitions
43	workforce_registry_entries	44	workforce_registry_versions

The Java implementation must preserve the later approved permission and audit registries in machine-readable contract files. Earlier draft counts must not override the latest checked-in reconciliation.

15. Module 2 domain rules

Concept	Rule
Person	Minimum-necessary identity; organization-scoped matching; no cross-tenant disclosure
Workforce member	Organization identity; may exist without login
Engagement	Effective employment/contract/visitor relationship; rehire creates new engagement
Practitioner	Regulated clinical identity; does not grant access, scope or assignment
Credential	Versioned and independently verified; evidence must be clean
Specialty	Regulated specialty distinct from organizational focus/capability
Scope	Independent clinical approval; restrictions and supervision explicit
Assignment	Effective facility/department/location placement; transfers preserve history
Service assignment	Links practitioner to service/context after eligibility checks
Availability	Working pattern only; not appointment capacity
Access	Existing governed roles and scopes; no parallel RBAC registry
Activation	Server-calculated gates; practitioner and non-clinical gate sets differ

15.1 Credential lifecycle

draft -> evidence_pending -> scanning -> submitted -> in_review -> verified | rejected | more_information_required | returned_for_correction. Renewal creates a superseding record. Suspension/revocation preserves authority, number, dates, evidence and decision history.

15.2 Scope lifecycle

draft -> submitted -> in_review -> approved | rejected | changes_requested -> suspended | superseded | ended. Submitter and approver differ in production. Eligibility recalculates after each material decision.

15.3 Assignment and offboarding

Transfers end/schedule the prior assignment and create a successor within one transaction. Offboarding ends engagement, assignments, service assignments and access according to effective time while preserving clinical attribution and evidence.

15.4 Expiry automation

Mutually exclusive expiry queues: 61-90, 31-60, 8-30, 0-7 and expired days. Notify at approved 90/60/30/7/0 milestones. Expiry invalidates future eligibility without rewriting historical care.

16. Module 3 - Patient Registry

ID	Screen
P3-01	Patient registry dashboard
P3-02	Patient directory
P3-03	Start registration
P3-04	Duplicate patient search
P3-05	Identity and demographics
P3-06	Contacts and addresses
P3-07	Communication preferences
P3-08	Identifiers
P3-09	Caregivers and proxies
P3-10	Consent and privacy
P3-11	Clinical safety flags
P3-12	Review and register
P3-13	Patient summary
P3-14	Potential duplicate queue
P3-15	Merge review
P3-16	Identity/audit timeline

16.1 Core entities

patient_profiles, patient_identifiers, patient_contacts, patient_addresses, communication_preferences, caregiver_relationships, patient_consent, privacy_restrictions, patient_safety_flags, patient_match_keys, patient_duplicate_candidates, patient_merge_requests, patient_merge_decisions, patient_registration_runs.

16.2 Registration rules

- Search before create using organization-scoped normalized identifiers, phone, email, name and date of birth.
- Never auto-merge; authorized reviewer resolves ambiguous duplicates.
- Record who supplied each datum and how it was verified.
- A patient can exist without portal access; portal invitation is separate.
- Caregiver access requires an explicit relationship, authority, effective dates and revocation path.
- Consent is purpose and version specific; withdrawal affects future use, not lawful historical record integrity.

17. Module 4 - Appointments and clinician assignment

ID	Screen
P4-01	Scheduling dashboard
P4-02	Calendar
P4-03	Appointment directory
P4-04	New appointment
P4-05	Patient selection
P4-06	Service/facility/location
P4-07	Eligible clinician selection
P4-08	Slot selection
P4-09	Appointment review
P4-10	Payment requirement
P4-11	Confirmation
P4-12	Reschedule
P4-13	Cancel/no-show
P4-14	Waitlist
P4-15	Appointment timeline

17.1 Core entities

schedules, slots, appointment_requests, appointments, appointment_participants, appointment_status_history, appointment_assignments, waitlist_entries, reminders, cancellations, no_show_decisions, appointment_payment_requirements, external_calendar_links.

17.2 Booking rules

- Service, location and practitioner eligibility are evaluated for the appointment instant.
- Slot holds are short lived and confirmed atomically to prevent double booking.
- Reschedule preserves the original appointment timeline.
- Cancellation/no-show policy is versioned and the applied version is recorded.
- WhatsApp/calendar/payment adapters cannot become the source of truth.

18. Module 5 - Encounters and episodes

ID	Screen
P5-01	Encounter dashboard
P5-02	Open encounter
P5-03	Patient/appointment context
P5-04	Participants
P5-05	Presenting concerns
P5-06	Clinical timeline
P5-07	Problems and diagnoses
P5-08	Orders and tasks
P5-09	Encounter notes
P5-10	Review/sign
P5-11	Amendment
P5-12	Encounter history

18.1 Core entities

episodes_of_care, encounters, encounter_participants, encounter_status_history, presenting_concerns, clinical_problems, diagnoses, encounter_notes, note_versions, orders, clinical_tasks, encounter_signatures, amendments.

18.2 Encounter lifecycle

planned -> arrived -> in_progress -> on_hold -> completed | cancelled | entered_in_error. Signed notes are append-only; correction is an amendment linked to the signed version. Participant identity, role, assignment and registration display context is snapshotted for historical attribution.

18.3 Safety

- Red flags create visible clinical tasks/escalations and cannot be dismissed silently.
- Only eligible practitioners may sign regulated clinical content.
- Late entries and amendments show author, time, reason and prior version.

19. Module 6 - Clinical Assessment and COS preservation

COS-01 through COS-27 are protected. Their exact names, copy, sequence, components and navigation must be imported from the original source package and mapped to production routes without visual redesign.

19.1 Production shell

Element	Requirement
Patient banner	Verified patient identity, alerts and encounter context
Responsible clinician	Current eligible RCP/doctor with assignment context
Step navigation	Preserve COS order and completion state
Autosave	Draft-safe, debounced, conflict aware and visible
Review	Section completeness, source and uncertainty
Sign-off	Clinician review, reason, timestamp and immutable version

19.2 Core entities

assessment_sessions, assessment_sections, assessment_responses, response_versions, clinical_narratives, assessment_instruments, instrument_versions, measurements, red_flags, assessment_reviews, assessment_signatures, assessment_amendments.

19.3 ROOTOPATHY clinical domains

The model must support conventional clinical information plus ROOTOPATHY dimensions and modalities without presenting speculative constructs as established medical fact. Each datum records source, method, unit, interpretation status and reviewer.

19.4 Monitoring principle

Every outcome measure has purpose, baseline, source, method, unit/scale, cadence, owner and action threshold.
Do not create a composite cure score.

20. Module 7 - Documents and results

ID	Screen
P7-01	Document dashboard
P7-02	Patient document list
P7-03	Upload document
P7-04	Classification/metadata
P7-05	Scan status
P7-06	Document viewer
P7-07	Result inbox
P7-08	Result detail
P7-09	Acknowledge/escalate
P7-10	Version history
P7-11	Export/share intent

20.1 Entities

documents, document_versions, document_links, document_classifications, document_scan_attempts, diagnostic_reports, lab_results, imaging_results, result_flags, result_reviews, result_escalations, document_access_intents.

20.2 Rules

- Use the private quarantine pipeline for every untrusted upload.
- Result values retain source, units, reference ranges, abnormal flags and provenance.
- Critical results require acknowledgement/escalation workflow with SLA evidence.
- Replacing a file creates a new version; never overwrite signed evidence.

21. Module 8 - AI assistance

ID	Screen
P8-01	AI session launcher
P8-02	Purpose and consent check
P8-03	Input selection
P8-04	Transcription/extraction
P8-05	Draft summary
P8-06	Clinical suggestion panel
P8-07	Safety/uncertainty flags
P8-08	Source/provenance viewer
P8-09	Clinician review/approval
P8-10	AI session history

21.1 Architecture

Spring Boot owns authorization, purpose, data minimization, workflow and final records. A separate Python service performs model-specific processing through versioned job contracts. Raw patient context is not placed on an uncontrolled message bus.

21.2 AI record

- Purpose, requester, tenant, patient, encounter and approved input manifest.
- Provider, model, model version, prompt/template version and parameters.
- Output schema, citations/provenance, uncertainty and safety flags.
- Reviewer edits, accept/reject decision, reason and final approved version.
- Latency, token/cost data, retries, failures and retention metadata.

21.3 Clinical governance

- AI output remains a draft and is visibly labeled.
- The clinician can edit, accept or reject; acceptance is never preselected.
- Prompt/model updates require evaluation datasets, safety sign-off and versioned rollout.
- Emergency red flags route to clinical escalation, not generative reassurance.

22. Modules 9 and 10 - Care planning, follow-up and outcomes

22.1 Module 9 screens

ID	Screen
P9-01	Care plan dashboard
P9-02	Create coordinated plan
P9-03	Problems/priorities
P9-04	Goals
P9-05	Interventions
P9-06	Modality coordination
P9-07	Owners/tasks
P9-08	Consent/preferences
P9-09	Safety/interaction review
P9-10	Clinician approval
P9-11	Patient summary
P9-12	Plan versions/amendments

22.2 Module 10 screens

ID	Screen
P10-01	Monitoring dashboard
P10-02	Rules
P10-03	Domains
P10-04	Measures
P10-05	Escalation
P10-06	Follow-up schedule
P10-07	Interpretation
P10-08	Confirm plan
P10-09	Outcome timeline

22.3 Entities

care_plans, care_plan_versions, goals, interventions, intervention_assignments, clinical_tasks, plan_consents, interaction_reviews, followup_plans, followup_events, outcome_definitions, outcome_measurements, escalation_rules, escalation_events, interpretations.

22.4 Rules

- Each intervention has rationale, owner, priority, start, review, stop criteria and monitoring.
- Cross-modality interaction/safety review is explicit.
- Patient goals and preferences are visible and versioned.
- Threshold breaches create owned escalation tasks and acknowledgement evidence.

23. Module 11 - Billing and payments

ID	Screen
P11-01	Billing dashboard
P11-02	Price books
P11-03	Packages
P11-04	Estimate
P11-05	Invoice
P11-06	Payment
P11-07	Payment link
P11-08	Refund/adjustment
P11-09	Claims
P11-10	Reconciliation
P11-11	Financial audit/export

23.1 Entities

price_books, price_items, packages, package_entitlements, estimates, invoices, invoice_items, payment_intents, payments, refunds, adjustments, claims, remittances, reconciliations, financial_exports.

23.2 Rules

- Clinical record completion is never altered because of payment status.
- CareOS never stores raw card data; payment provider owns regulated card entry.
- Webhooks are signed, idempotent and reconciled against expected amount/currency.
- Refund and adjustment require reason, authorization and immutable financial audit.

24. Modules 12 and 13 - Reporting, governance and integrations

24.1 Module 12 screens

ID	Screen
P12-01	Reporting dashboard
P12-02	Operational reports
P12-03	Clinical safety reports
P12-04	Outcome reports
P12-05	Workforce governance
P12-06	Access/security reports
P12-07	AI governance
P12-08	Financial reports
P12-09	Scheduled exports
P12-10	Report audit/history

24.2 Module 13 screens

ID	Screen
P13-01	Integration dashboard
P13-02	FHIR endpoints
P13-03	Terminology mappings
P13-04	WhatsApp messaging
P13-05	Payment configuration
P13-06	Calendar integration
P13-07	Lab/imaging interfaces
P13-08	Webhook management
P13-09	Mobile/API clients
P13-10	Integration audit/replay

24.3 Reporting controls

- Every report applies tenant/resource policies and minimum-necessary fields.
- Sensitive exports require purpose, recent authentication and expiring private delivery.
- CSV generation prevents spreadsheet formula injection.

24.4 Integration controls

- Inbound webhooks require signature, replay protection and idempotency.
- Outbound events are versioned and delivered from the transactional outbox.
- Failed messages enter monitored retry/dead-letter workflows with safe replay.

25. FHIR interoperability baseline

CareOS domain	FHIR baseline mapping
Organization	Organization, Location, HealthcareService
Workforce	Practitioner, PractitionerRole
Patient	Patient, RelatedPerson, Consent
Scheduling	Schedule, Slot, Appointment
Care delivery	Encounter, EpisodeOfCare, Condition, Observation
Assessment	Questionnaire, QuestionnaireResponse, ClinicalImpression
Documents/results	DocumentReference, DiagnosticReport, Media
Care plan	CarePlan, Goal, ServiceRequest, Task
Governance	Provenance, AuditEvent

25.1 Boundary rules

- FHIR is an external representation, not the internal database schema.
- Maintain explicit versioned mapping and terminology services.
- Validate resources/profiles and record provenance for import/export.
- Use idempotent conditional create/update where supported.
- A sustained FHIR workload may move to a separate Java/HAPI FHIR service.

25.2 Initial API posture

Start with purpose-specific integration endpoints and approved FHIR profiles. Do not expose a broad generic FHIR server until consent, search scope, terminology, audit and data-residency policies are implemented and tested.

26. REST API contract

26.1 Conventions

Concern	Contract
Base	/api/v1/organizations/{organizationId}/...
IDs	Opaque UUIDv7; never derive authorization from UUID secrecy
Errors	application/problem+json using RFC 9457 with stable error codes
Pagination	Opaque cursor; limit capped; stable sort plus ID tie-breaker
Filtering	Allow-listed server filters; query string persists UI state
Concurrency	ETag/If-Match or expectedLockVersion; 409/412 conflict
Retry	Idempotency-Key for create/payment/upload/approval retry paths
Time	RFC3339 UTC instants; IANA timezone identifiers
Validation	Stable field pointers and accessible summary
Correlation	Request/correlation ID returned and logged safely

26.2 Example problem detail

```
{
  "type": "https://careos.example/problems/stale-revision",
  "title": "Record changed",
  "status": 409,
  "code": "record.stale_revision",
  "detail": "Reload the latest record before saving again.",
  "correlationId": "..."}

```

26.3 API documentation

- OpenAPI is generated/validated in CI and stored in contracts/openapi.
- Frontend types/client are generated from the same approved contract.
- Breaking changes require a new API version or documented compatibility path.

27. State machines

Aggregate	Baseline states
Organization	draft -> submitted -> active -> suspended -> closed
Facility	draft -> active -> temporarily_suspended -> active closed
Workforce	draft -> pending_review -> active -> suspended -> active offboarded
Credential	draft -> scanning -> submitted -> in_review -> verified rejected information_required -> superseded/revoked
Scope	draft -> submitted -> in_review -> approved rejected changes_requested -> suspended/superseded/ended
Appointment	requested -> held -> confirmed -> arrived -> fulfilled cancelled no_show
Encounter	planned -> arrived -> in_progress -> on_hold -> completed cancelled entered_in_error
Document	quarantined -> scanning -> clean/protected infected rejected failed
AI session	created -> queued -> running -> draft_ready -> under_review -> approved rejected failed
Care plan	draft -> review -> approved -> active -> revised completed cancelled
Invoice	draft -> issued -> partially_paid -> paid void written_off; refund tracked separately

27.1 Transition contract

Every transition defines allowed source state, permission, tenant/resource scope, reason requirement, MFA/recent-auth requirement, maker-checker rule, effective time, revision expectation, database invariant, canonical audit event, outbox event and user-visible success/failure behavior.

27.2 Delete policy

Only unreferenced drafts may be physically deleted. Effective, approved, signed, paid, clinically attributed or audited records are corrected, amended, superseded, ended, revoked or archived.

28. Validation catalogue

Category	Examples
Identity	Normalized identifiers; realistic date; country-specific formats; duplicate search first
Effective dates	Start before end; containment within engagement; no prohibited overlap
Hierarchy	Department/location belongs to selected facility and organization
Professional	Authority, jurisdiction, profession, issue/expiry, restriction and evidence consistency
Availability	Multiple intervals, overnight explicit, timezone valid, no overlap
Appointment	Slot current, eligibility current, no double booking, patient active
Clinical	Required units/source/method, red-flag response, signer eligibility
Documents	Size, MIME, magic bytes, hash, scan state and purpose
AI	Approved purpose, consent/legal basis, schema, model/prompt version and human reviewer
Finance	Currency/amount consistency, signed webhook and idempotent reconciliation

28.1 Validation response

- Return field-specific errors plus a stable top-level problem code.
- Frontend places focus on the accessible error summary and links to controls.
- Never rely only on browser validation or hidden disabled controls.
- Localized display does not change canonical API formats.

29. Security and privacy model

Threat	Control
Cross-tenant access	Forced RLS, composite FKs, tenant-bound routes and attack tests
Privilege escalation	Delegation ceiling, maker-checker, final-owner guard and access diff preview
Credential theft	MFA, session rotation, short privileged sessions and secret manager
Injection	Parameterized queries, validation, output encoding and CSP
Malicious upload	Quarantine, integrity checks, fail-closed malware scan and private promotion
Sensitive export	Purpose, scope, recent auth, expiring private artifact and audit
AI disclosure	Minimum data, approved provider, retention controls and human review
Webhook replay	Signature, timestamp/nonce, idempotency and reconciliation
Insider access	Least privilege, time-limited elevation, monitoring and audit review
Data loss	Encrypted backups, point-in-time recovery and restore exercises

29.1 Privacy-by-design

- Purpose and legal basis are recorded for sensitive processing.
- Data minimization and field-level sensitivity classifications guide views/exports.
- Retention/disposal policies are jurisdiction and record-type specific.
- Patient access, correction and consent workflows must be reviewed by qualified legal/privacy owners.

This document is technical guidance, not a legal compliance opinion. Production deployment requires jurisdiction-specific clinical, privacy, security and legal review.

30. Frontend and Design System requirements

30.1 Visual foundation

Area	Rule
Brand	ROOTOPATHY CareOS; preserve approved wordmark and workspace naming
Palette	Dark clinical green foundation, layered green surfaces and mint interaction accent
Navigation	Workspace header plus grouped sidebar; responsive drawer on smaller screens
Cards	Subtle border, consistent radius, no generic admin-template styling
Forms	Persistent labels, help/error text, clear focus, accessible summaries
Tables	Server data, sortable headers, responsive card fallback and cursor pagination
Status	Text/icon plus color; no color-only meaning
Dialogs	Focused purpose, confirmation/reason, focus trap and return focus

30.2 Required reusable components

AppShell, WorkspaceHeader, OrganizationSwitcher, SidebarNav, MobileNav, PageHeader, Breadcrumbs, Card, Button, TextField, SelectField, DateField, TimeField, RadioGroup, Checkbox, FileDropzone, DataTable, ResponsiveCardList, FilterGrid, CursorPager, Badge, Alert, Toast, Skeleton, EmptyState, ErrorSummary, ConfirmReasonDialog, RecentAuthDialog, MfaChallenge, ApprovalPanel, EvidenceViewer and Timeline.

30.3 Responsive and accessibility

- Test at 1440, 1024, 768, 390 and 320 pixels.
- No horizontal page overflow, label/input collision or inaccessible sticky control.
- Keyboard operation, visible focus, landmarks, labels, descriptions and aria-live status.
- WCAG 2.2 AA target, 44px touch targets where applicable and reduced-motion support.
- Loading, empty, no-result, validation, denied, stale, failure and success states for every screen.

31. Jobs, notifications and scheduling

Job family	Examples	Reliability
Documents	Scan, promotion, thumbnail/preview	Unique per document/version; bounded retry
Credentialing	Expiry milestones, eligibility re-evaluation	Tenant context; deduplicate milestone
Appointments	Reminders, waitlist offers, calendar sync	Outbox/event driven
Clinical	Follow-up due, escalation SLA	Owned task and acknowledgement
AI	Transcription, extraction, synthesis	Timeout, cancellation, circuit breaker
Exports	CSV/PDF generation and expiry	Private artifact; purpose-scoped
Integrations	Webhook delivery/replay	Versioned payload, idempotent consumer
Retention	Policy disposal/legal hold	Dry-run, approval and immutable evidence

31.1 Notification abstraction

Email is initial. SMS and WhatsApp are adapters. Store template/version, recipient reference, purpose, channel, attempt, provider ID, status and timestamps. Never put sensitive clinical detail in an unapproved notification channel.

31.2 Scheduler safety

- Distributed lock prevents duplicate scheduler execution.
- Each item remains idempotent even if the scheduler runs twice.
- Backlogs, retry exhaustion and SLA breaches create operational alerts.

32. Observability and operations

Signal	Required view
API	Rate, p50/p95/p99 latency, errors and saturation by route family
Database	Connections, slow queries, locks, replication and storage
Redis/jobs	Queue depth, age, retries, failures and worker health
Documents	Upload/scan latency, rejection and scanner availability
AI	Latency, error, provider/model, cost and safety/review outcomes
Business	Registration, booking, encounter, result acknowledgement and follow-up SLA
Security	Auth failures, MFA, elevation, exports, cross-tenant denials and anomalies

32.1 Logging

- Structured JSON logs with correlation ID, safe actor/resource references and environment.
- No passwords, tokens, document URLs, raw health content or unnecessary PII.
- Audit is a domain evidence store; application logs are not a substitute.

32.2 Runbooks

Startup, deployment, rollback, migration, backup/restore, scanner outage, queue backlog, database incident, credential compromise, privacy incident, AI provider outage and clinical-safety escalation.

33. Test architecture and quality gates

Layer	Tool/coverage
Unit	JUnit 5 for domain rules and transitions
Integration	Spring Boot Test + Testcontainers PostgreSQL/Redis/object storage
Architecture	ArchUnit module dependency rules
Database	Flyway migration, RLS, constraint, trigger and cross-tenant attack tests
Contract	OpenAPI compatibility, permissions, authorization clauses, audit events
Frontend	Vitest and Testing Library
Browser	Playwright critical workflows at desktop/tablet/390/320
Accessibility	Axe plus keyboard/focus/manual verification
Security	Dependency, secret, SAST, container and upload-pipeline scans
Performance	API/search/load tests using synthetic data

33.1 Test isolation

- Automated tests use careos_test only and fail fast if configured otherwise.
- Testcontainers use disposable databases; no destructive commands against development.
- Browser, storage, Redis, email and scanner adapters are isolated by environment.
- Record careos_dev counts/checksums before and after full suites and fail on change.

33.2 Release gate

Formatting, compilation, static analysis, complete tests, migrations, accessibility, production builds, dependency audit, secret scan, image scan, COS checksum and protected-module regression must all pass. No critical workflow skip is acceptable.

34. Local development and configuration

34.1 Environment variables

Group	Examples
Application	CAREOS_ENV, CAREOS_BASE_URL, SPRING_PROFILES_ACTIVE
Database	DB_URL, DB_USERNAME, DB_PASSWORD, DB_POOL_SIZE
Redis	REDIS_URL, REDIS_PREFIX
Security	SESSION_SECRET/KEY references, MFA encryption key reference, allowed origins
Storage	S3 endpoint, region, quarantine bucket, protected bucket
Scanner	Scanner adapter and credentials; production required
Email	SMTP/provider configuration and sender
AI	AI service URL, provider secret references, timeout and data policy

Only variable names and safe local placeholders belong in .env.example. Real .env files, passwords, tokens, private keys and database dumps never enter Git.

34.2 Local commands

```
git clone <repository> careos-java
cd careos-java
copy .env.example .env # Windows; use cp on macOS/Linux
docker compose up -d postgres redis minio mailpit
cd backend
./mvnw spring-boot:run
# separate terminal
cd frontend
npm ci
npm run dev
```

34.3 Full container startup

```
docker compose build
docker compose up -d
docker compose ps
```

35. CI/CD and deployment

35.1 Pull-request pipeline

- Secret scan and forbidden-file check.
- Maven format/checkstyle, compile, tests, ArchUnit and dependency scan.
- Flyway migration and PostgreSQL RLS/constraint tests in Testcontainers.
- npm ci, TypeScript strict, ESLint, Prettier, Vitest and production build.
- Playwright/Axe critical flows and responsive views.
- OpenAPI, permission and audit-event contract verification.
- Container build, SBOM and image vulnerability scan.

35.2 Deployment sequence

1. Build immutable signed images and frontend assets.
2. Back up and verify database readiness.
3. Run backward-compatible Flyway migrations as a controlled job.
4. Deploy API/workers with health, readiness and startup probes.
5. Deploy frontend assets and verify version compatibility.
6. Run smoke tests and tenant-isolation probe.
7. Monitor error, latency, queue and business metrics.
8. Rollback application safely; use forward database repair rather than destructive rollback where required.

35.3 Production hosting

Do not treat basic shared hosting as the target for this Java architecture. Use a platform that supports OCI containers, managed PostgreSQL, Redis, private object storage, background workers, scheduler, TLS, monitoring, backups and secret management.

36. Implementation phases and acceptance

Phase	Release	Completion scope
0	Foundation	Repository, CI, Design System, API conventions, tenant/RLS, auth, audit/outbox
1	Administration	M1-01 through M1-23 and organization activation
2	Workforce	M2-01 through M2-29 and production closeout
3	Patient	Registration, consent, proxy and duplicate governance
4	Scheduling	Appointments, availability consumption and assignment
5	Encounters	Episodes, encounters, notes and amendments
6	Assessment	Production COS-01 through COS-27
7	Documents	Clinical documents and results
8	AI	Python service integration and clinician approval
9	Care/outcomes	Plans, follow-up, monitoring and escalation
10	Finance/reporting	Billing and governed reporting
11	Integrations	FHIR, messaging, payment, labs, calendar and mobile

36.1 Phase acceptance

- Approved architecture and policy decisions.
- Approved clickable high-fidelity mockups.
- Traceability from requirement to screen, API, permission, event and tests.
- Production persistence and all states/actions connected.
- Full security, accessibility, browser and regression evidence.
- Owner accepts PASS before commit/freeze tag.

37. Codex master bootstrap prompt

```
Build the CareOS Java + React/Node project from scratch using
CareOS_Complete_Build_Specification_Java_Spring_Boot_React_Node_Edition.pdf as the architecture authority and the supplied approved
CareOS Design System, M1/M2 mockups/prototypes and COS-01 through COS-27 assets as visual/workflow authorities. First read every
source completely, inspect AGENTS.md and Git status, then report assumptions, decisions, dependency graph, proposed repository tree,
security model, database/RLS approach, implementation sequence and genuine blockers. Do not implement all modules at once. Implement
Phase 0 foundation only: Java 25 LTS, Spring Boot 4.1.x modular monolith, Maven wrapper, React 19.3 with strict TypeScript using
Node 24 LTS, PostgreSQL 18 with Flyway and forced RLS, Redis/Valkey, private S3-compatible storage ports, secure same-origin
sessions/CSRF, MFA/invitation abstractions, canonical permission and audit registries, transactional outbox, idempotency, optimistic
locking, OpenAPI/RFC 9457, Docker Compose, CI, synthetic-only deterministic seeds and complete test isolation. Create no clinical
pages and do not alter supplied mockups/assets. Use careos_test only for automated tests. Add JUnit/Testcontainers, ArchUnit,
Vitest, Playwright and Axe foundations. Run all Phase 0 gates and produce a gap ledger plus evidence-based PASS or FAIL. Do not
commit or tag; stop for approval after Phase 0.
```

37.1 Why this prompt stops after foundation

A single big-bang Codex request creates false completion risk. Foundation must prove tenant isolation, authorization, transaction/audit behavior and reproducible builds before screens are added.

38. Codex Module 1 implementation prompt

Continue from the verified Java Phase 0 foundation. Implement CareOS Module 1 M1-01 through M1-23 exactly from the approved administration mockups and this build specification. Preserve the CareOS Design System and COS assets. Implement organization, memberships, governed RBAC, facilities, departments, locations, operating hours, holidays, services, facility/location service assignments, identifier schemes, administrator invitations/access, setup readiness, activation, configuration history and audit. Use real PostgreSQL persistence, forced RLS, composite tenant FKs, server validation, optimistic locking/idempotency, canonical audit and transactional outbox. Enforce final active Organization Owner safeguards at UI, policy, locked transaction and database levels. New facilities start as draft; closure fields are conditional. Weekly operating-hours save creates one parent configuration version with child resource changes. Activation validation returns inline typed results with run ID/time/gates and maker-checker approval. Build all loading/empty/validation/denied/conflict/failure/success states. Test desktop/tablet/390/320, keyboard, Axe, cross-tenant attacks and careos_test isolation. Run every quality gate and report M1-01 to M1-23 PASS/FAIL. Do not commit/tag or begin Module 2.

39. Codex Module 2 implementation prompt

Continue from the approved Java Module 1 freeze. Implement CareOS Module 2 M2-01 through M2-29 using the exact approved final mockups and the 44-table schema in this specification. Keep Person, User, Workforce Member, Engagement, Practitioner, Credential, Scope, Assignment, Service Assignment, Availability and Access independent. Reuse the single users table and governed Module 1 RBAC; never create parallel role/permission tables. Implement clinical and non-clinical onboarding, organization-scoped duplicate matching, effective-dated engagement, practitioner/qualification/registration, private credential upload/quarantine/fail-closed scanning, independent credential review, specialties, versioned scope/maker-checker, hierarchy assignments/transfers, service eligibility, scoped roles, atomic weekly availability, optional account linkage/invitations/MFA, readiness/activation, profile, suspend/reactivate, offboarding, expiry/reminders, configuration history, audit/export, controlled registries and member evidence timeline. Every action must enforce tenant scope, forced RLS, permission, validation, transaction, revision/idempotency, reason, MFA/recent-auth/maker-checker where required, canonical audit and outbox. Implement server search/filter/sort/cursor pagination and purpose exports. Add JUnit/Testcontainers, frontend, Playwright, Axe and 320px coverage for every critical path. Validate the latest machine-readable permission/authorization/audit registries without relying on superseded draft counts. Report a complete screen ledger and PASS/FAIL. Do not commit/tag or begin Module 3.

40. Codex prompt template for Modules 3-13

Create the architecture and product-definition package for CareOS Module [NUMBER - NAME] only. Use the Java master build specification, all frozen previous modules, CareOS Design System and any supplied mockups as authority. Do not write production code. Define purpose, boundaries, dependencies, actors, full user journey, screen register, navigation, field-level forms, validations, empty/loading/denied/conflict/failure/success states, database entities/relationships, effective-date and retention rules, state machines, route/API contracts, permissions/scopes, MFA/recent-auth/maker-checker requirements, canonical audit/outbox events, jobs/notifications, security/privacy/clinical-safety controls, reporting, FHIR mappings where applicable, synthetic seed scenarios, acceptance tests and responsive/accessibility requirements. Identify genuine policy decisions. Produce high-fidelity clickable mockups only after product approval. Preserve all frozen modules and COS assets. Stop before production implementation.

40.1 Implementation prompt template

Implement approved CareOS Module [NUMBER - NAME] on the verified Java foundation using its final specification and clickable mockups. First produce a COMPLETE/PARTIAL/MISSING gap ledger and traceability matrix. Implement dependency-ordered vertical slices with real PostgreSQL persistence, forced RLS, server validation, transactions, optimistic locking/idempotency, canonical audit/outbox, secure document/AI boundaries and governed lifecycle actions. Use generated OpenAPI types in React; no client-only production actions or hard-coded production data. Verify JUnit/Testcontainers, ArchUnit, frontend unit tests, Playwright critical workflows, Axe, 390/320 layouts, builds, dependency/security scans, previous-module regression, test database isolation and COS checksum. Provide exact evidence and PASS/FAIL. Do not commit/tag or begin another module.

41. Codex final verification, checkpoint and ZIP prompts

41.1 Final verification

Perform final CareOS [MODULE/RELEASE] closeout. Do not redesign or add scope. Reconcile every screen, route, table, permission, authorization clause, audit event, job and acceptance test against the authoritative specification. Resolve every real gap. Run formatting, compilation, static analysis, complete backend/frontend/integration/browser/accessibility/security/performance/build gates, RLS attack tests, careos_test isolation, careos_dev before/after proof, frozen-module regressions and COS checksum. Provide exact outputs and evidence-based PASS or FAIL. Do not claim PASS for client-only, unreachable, insecure, untested or skipped critical behavior. Do not commit or tag.

41.2 Safe checkpoint

After verified PASS and explicit approval, inspect complete Git status/diff. Stage only reviewed source, migrations, contracts, safe configuration, documentation and maintained tests. Exclude node_modules, Maven target, frontend dist/build, reports, caches, logs, temporary files, dumps, real credentials and unapproved artifacts. Show exact staged files, run diff check and secret scan, then create one normal commit with the approved message. Do not create/move/delete a tag. Report commit hash, parent, included/excluded files, remaining tree and protected baseline proof.

41.3 Safe source ZIP

Create a source-only CareOS Java + React/Node ZIP from the approved clean Git commit. First verify HEAD, status, tags and secret scan. Use git archive so only tracked committed source is packaged. Exclude secrets, real .env, node_modules, backend target, frontend dist, reports, caches, logs, dumps, test artifacts and real data. Include README, .env.example, Maven wrapper, package lock, compose file, migrations, contracts, docs and tests. Generate SHA-256 checksum, list archive contents and verify it extracts. Do not modify source or create a tag. Report the ZIP path, size, checksum and exact commit.

42. Traceability and Definition of Done

Requirement	Required trace
Screen action	Mockup -> route -> permission -> handler -> service -> database -> audit/outbox -> tests
State transition	Source state -> validation -> step-up control -> target state -> evidence
Sensitive view/export	Purpose -> scope -> RLS -> field policy -> audit -> expiry
External message	Domain event -> outbox -> adapter -> attempt -> acknowledgement/retry
AI output	Purpose/input -> model/prompt version -> output/safety -> human review -> final record
FHIR exchange	Canonical entity -> mapping/version -> validation -> provenance -> outcome

42.1 A module is done only when

- Architecture, policies and mockups are approved.
- Every visible action is real, authorized and persisted.
- All lifecycle/history/retention rules are enforced at the appropriate layers.
- All UI states and responsive/accessibility requirements pass.
- All contract, RLS, security, audit/outbox and regression tests pass.
- Operational configuration and runbooks exist; external gaps fail closed.
- Owner accepts evidence-based PASS and authorizes the freeze checkpoint.

43. Open decisions before production

Decision	Owner group
Jurisdiction-specific privacy, retention, residency and legal-hold policy	Legal/privacy/security
Patient identity proofing and cross-organization matching lawful basis	Privacy/product
Profession/service credential and supervision matrices	Clinical governance
Break-glass access design and retrospective review	Clinical safety/security
AI providers, regions, retention, training prohibition and evaluation thresholds	AI governance/privacy/clinical
Critical-result and red-flag response SLAs	Clinical governance/operations
Notification channels, consent and escalation sequence	Product/privacy/operations
Appointment cancellation/no-show/refund policy versions	Operations/finance/legal
Production hosting, RPO/RTO and monitoring budget	Executive/technology
Java team staffing, ownership and long-term support	Executive/technology

43.1 Recommended immediate action

Approve this Java architecture decision, assemble the required input package, and run the Phase 0 Codex bootstrap prompt. Do not ask Codex to generate all modules in one uninterrupted task.

44. Official technical references

Reference	URL	Use
Spring Boot system requirements	https://docs.spring.io/spring-boot/system-requirements.html	4.1.1 stable; Java 17 minimum and compatibility through Java 26 at verification time
Oracle Java SE support roadmap	https://www.oracle.com/java/technologies/java-se-support-roadmap.html	Java 25 is LTS
Node.js releases	https://nodejs.org/en/about/previous-releases	Node 24 is LTS; production should use Active/Maintenance LTS
React versions	https://react.dev/versions	React 19.3 documentation at verification time
PostgreSQL release notes	https://www.postgresql.org/docs/current/release.html	PostgreSQL 18 current at verification time
Redis documentation	https://redis.io/docs/latest/	Redis current documentation and operations
Spring Security	https://docs.spring.io/spring-security/reference/	Authentication and authorization reference
Spring Modulith	https://docs.spring.io/spring-modulith/reference/	Modular application boundary guidance
HAPI FHIR	https://hapifhir.io/hapi-fhir/docs/	Optional Java FHIR boundary

44.1 Accuracy note

Framework patch versions and security advisories change. At repository bootstrap and each release, re-check official compatibility and lock exact versions through Maven and npm lockfiles. This specification pins the architecture and major version baseline, not permission to upgrade without tests.

44.2 Final handoff statement

This Java edition contains the information required to initialize CareOS from scratch, define the full product structure and direct Codex through controlled implementation. It does not replace the original visual assets, clinical policy approvals or module-specific final mockups. Those inputs are mandatory parts of the build package.